

Rural KV

Prototipo Funcional Semana 10
(SW303 LP-1, UNI-FIIS)

Motor de Base de Datos Clave-Valor en C Puro

Integrantes:

Piñas Camarena Aldair Samir
Santiago Ayala Luis Angel
Puertas Flores Daniel Efrén
Monje Maron Alan Josue

Docente:

Mag. Juan Luis Herencia Guerra

Restricciones del Entorno: Por qué Offline-First

DIAGNOSTICO_DEL_CONTEXTO

Brecha Digital:

Solo 21.7% cuenta con acceso con acceso a internet fijo en áreas rurales.



Hardware Restringido:
Despliegue en Edge/IoT con extrema limitación energética.

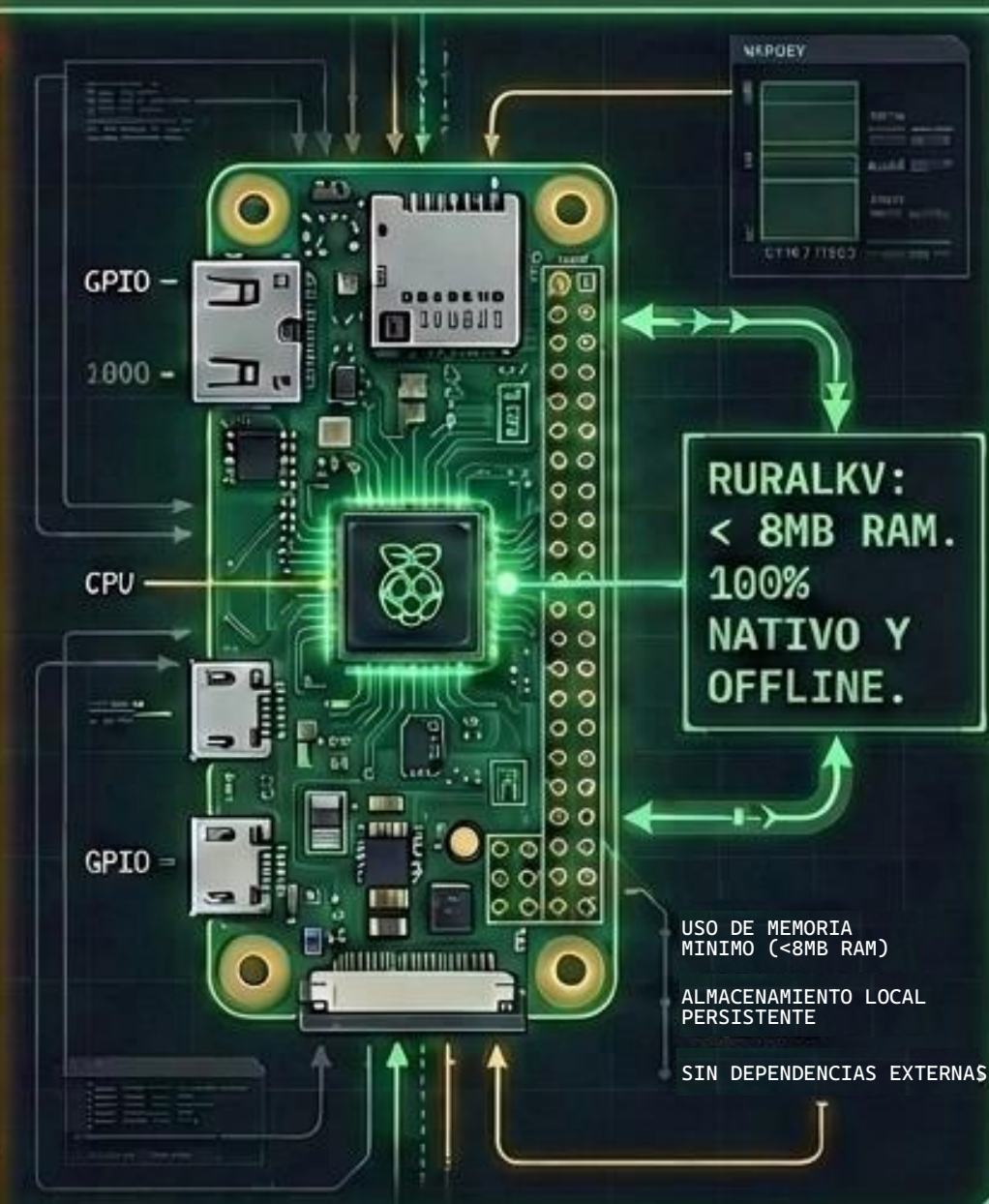
BASES TRADICIONALES
(Redis/SQLite)



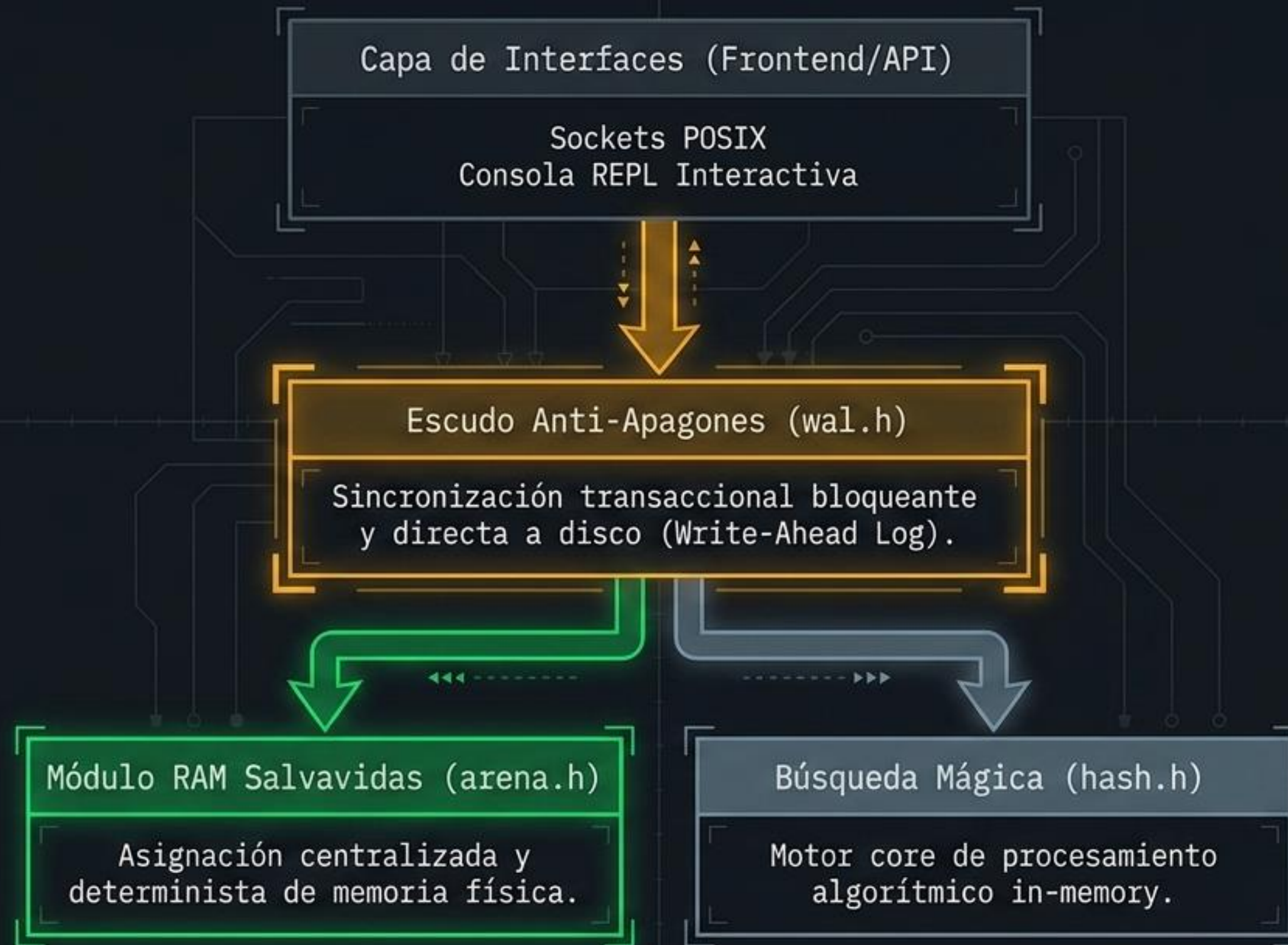
RECHAZADO_POR_ARQUITECTURA

- ⚠ REQUIERE > 100MB RAM
- ⚠ SOBRECARGA RELACIONAL

RURALKV: ARQUITECTURA OPTIMIZADA



Arquitectura General y Flujo de Procesamiento del Motor



Diseño Nativo: La Estructura Node y Expiración TTL

```
>_  
  
typedef struct Node {  
    char* key;  
    char* value;  
    time_t expiry;  
    struct Node* next;  
} Node;
```

Estructura Atómica:

Uso de punteros dinámicos hacia arrays de caracteres en la Arena.

Expiración Determinista:

Integración con `<time.h>` para barrido de claves caducadas en $O(1)$.

Topología Interna:

Puntero circular para enrutamiento de listas enlazadas (Resolución de colisiones).

Motor Hashing: Complejidad **0(1)** con Algoritmo DJB2



Eficiencia de Hardware: Conversión de cadenas alfanuméricas hacia índices espaciales procesada teóricamente en microsegundos.

Integridad de Datos: Resolución de Colisiones por Chaining

Principio del Palomar Computacional



Topología de Listas Enlazadas:

Si `hash_put()` detecta ocupación previa, instancia dinámicamente un nuevo nodo y actualiza la cabecera desplazando los punteros sobre el atributo `Node* next`, garantizando que no se sobrescriba información crítica en producción.



```
rural-kv> SET DNI_01 Gripe_Alta
```

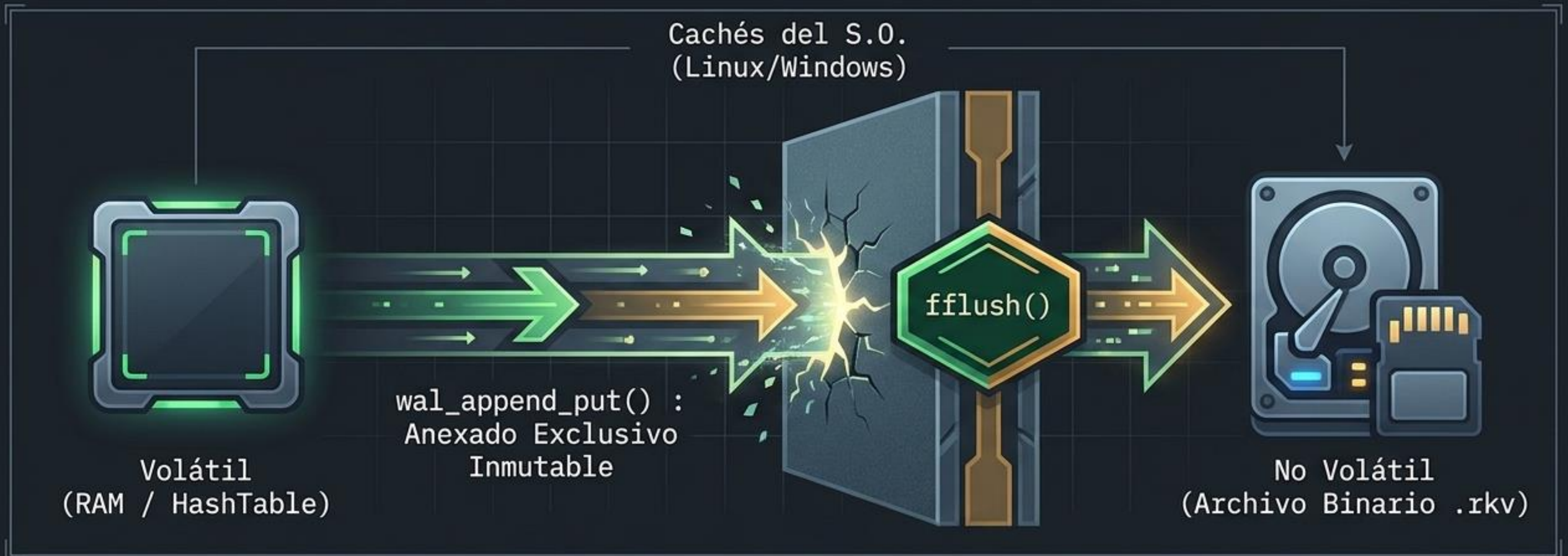
A diagrammatic line connects the 'SET' command in the terminal to the first row of the table on the right. The line starts from the 'SET' command, goes down, then right, then up, ending with an arrow pointing to the first row.

Parseo Léxico Dinámico:
Captura de texto y separación vía
comando `split(' ', 2)`

1.	SET key value	: Indexación algorítmica y almacenamiento en memoria RAM.
2.	GET key	: Búsqueda eficiente $O(1)$ con retorno inmediato del puntero.
3.	DEL key	: Liberación lógica y remoción del nodo en la cubeta hash.
4.	KEYS *	: Listado dinámico y recorrido de la totalidad de claves vigentes en la HashTable.
5.	TTL key	: Consulta y lectura de la marca de tiempo de expiración mediante <code><time.h></code> .
6.	SAVE	: Serialización explícita y persistencia transaccional directa a archivo binario (.rkv).

Persistencia de Datos y Resiliencia

Write-Ahead Log (WAL)



Evasión Forzosa de Cachés: No confiamos en la memoria diferida del sistema. La instrucción `fflush()` fuerza la escritura perentoria de la transacción sobre la placa física antes de alterar la memoria RAM, burlando pérdidas por fallos eléctricos.

Gestión de Memoria Segura, Pruebas y Aseguramiento de Calidad

The Architecture



The Validation



Conclusiones del Sprint Semana 10 y Próximos Pasos

Hitos Semana 10

- ❑ **Motor Core C11:** Complejidad $O(1)$ comprobada.
- ❑ **Contención RAM:** < 8MB verificado en Raspberry Pi.
- ❑ **Persistencia WAL** y Cero Fugas de Memoria.

- ❑ **Concurrencia Multi-Cliente:** Integración de hilos pthreads sobre /sync.
- ❑ **Algoritmo Delta Sync:** Sincronización diferencial sobre Sockets.
- ❑ **Sistema Pub/Sub:** Módulo local ultra-ligero nativo.

Entrega Final (Semana 14)

